Practical AI for Academics

Session 1 – an introduction to Generative AI

Claes Bäckman
Leibniz Institute for Financial Research SAFE

Let’s start with an example – building a website for a paper

Three messages

1. **The tools changed.** Chat gave way to agents that read files, run code and iterate. The tool you tried in 2023 is not the tool of today.
2. **The workflow is the value.** The agent's leverage comes from context (your files), standing instructions, and reusable skills — not from raw model IQ.
3. **Judging the output is the new job.** Silent errors and confident statements with no backing are what you should look out for.

Workshop synthesis, drawing on Mollick (2026), Velikov (2026) and Korinek (2025).

**AI should let us do new things,
not only old things faster**

Seven steps now, hands-on later

1	2	3	4	5	6	7
What an LLM is	**Landscape**	**Use cases**	**Teaching**	**Tools and context**	**Verification**	**What this means**
Prediction, not lookup. Confident when wrong.	Three model generations; who uses them.	Research, coding, admin, communication.	Protect learning, then teach more.	The harness, the window, choosing a tool, a prompt to copy.	Checks for silent errors.	Friction, bottlenecks, norms.

Session 2. Hands-on with your own materials: a referee report on your draft, secure data, writing, the paper–code link.

Slides, skills and guides for both days: claesbackman.com/ai_workshop.

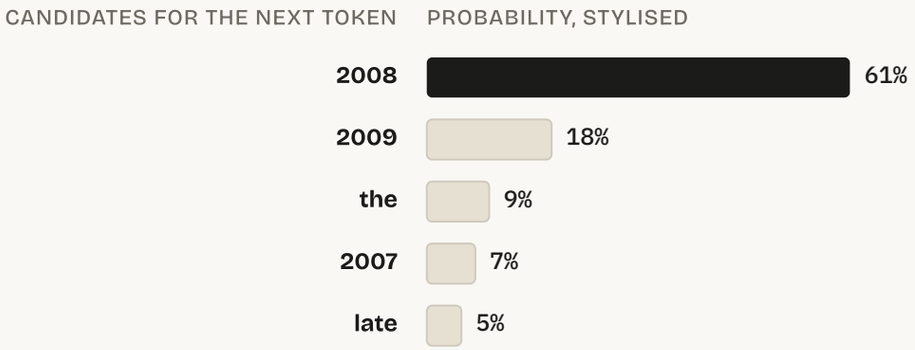
1 · An LLM predicts the next token



An LLM predicts the next token, over and over, very well

A token is a word or a piece of a word, about four characters of English. That is the whole training objective: given the text so far, predict the next token. Press the button.

```
House prices in Denmark fell sharply in
```



Sample the next token | Reset | Press **sample**: the most likely token is appended, then the model predicts again.

Candidate lists and probabilities are invented for the slide; a real model has tens of thousands of candidates. The mechanism is the real one: predict a distribution over the next token, pick one, repeat. The stage always picks the top candidate; a real model samples.

So it is a prediction machine, not a lookup machine

Not a database

It does not look facts up. It generates text that is plausible given what it has seen.

Not a calculator

It predicts what an answer looks like. The arithmetic is right when the pattern is common, and the tools in section 5 exist because of this.

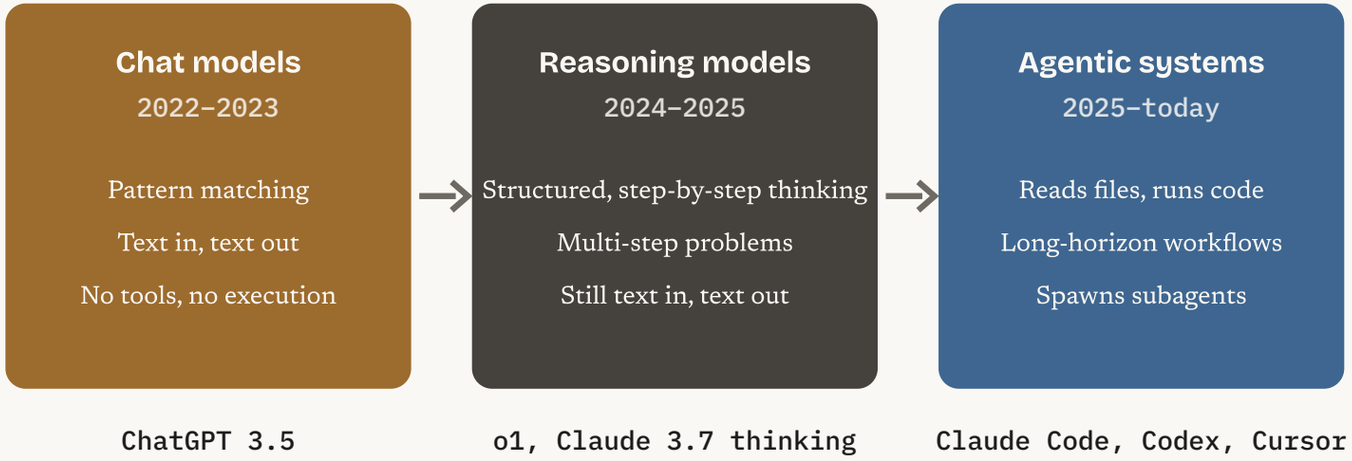
Not connected to your files

Or to the live web, unless the tool around it adds that. Out of the box it knows nothing about your paper.

A very well-read, very fast colleague who has read almost everything but remembers none of it precisely, has no access to your files, will never admit to not knowing, **and is** unapologetic about being wrong**.**

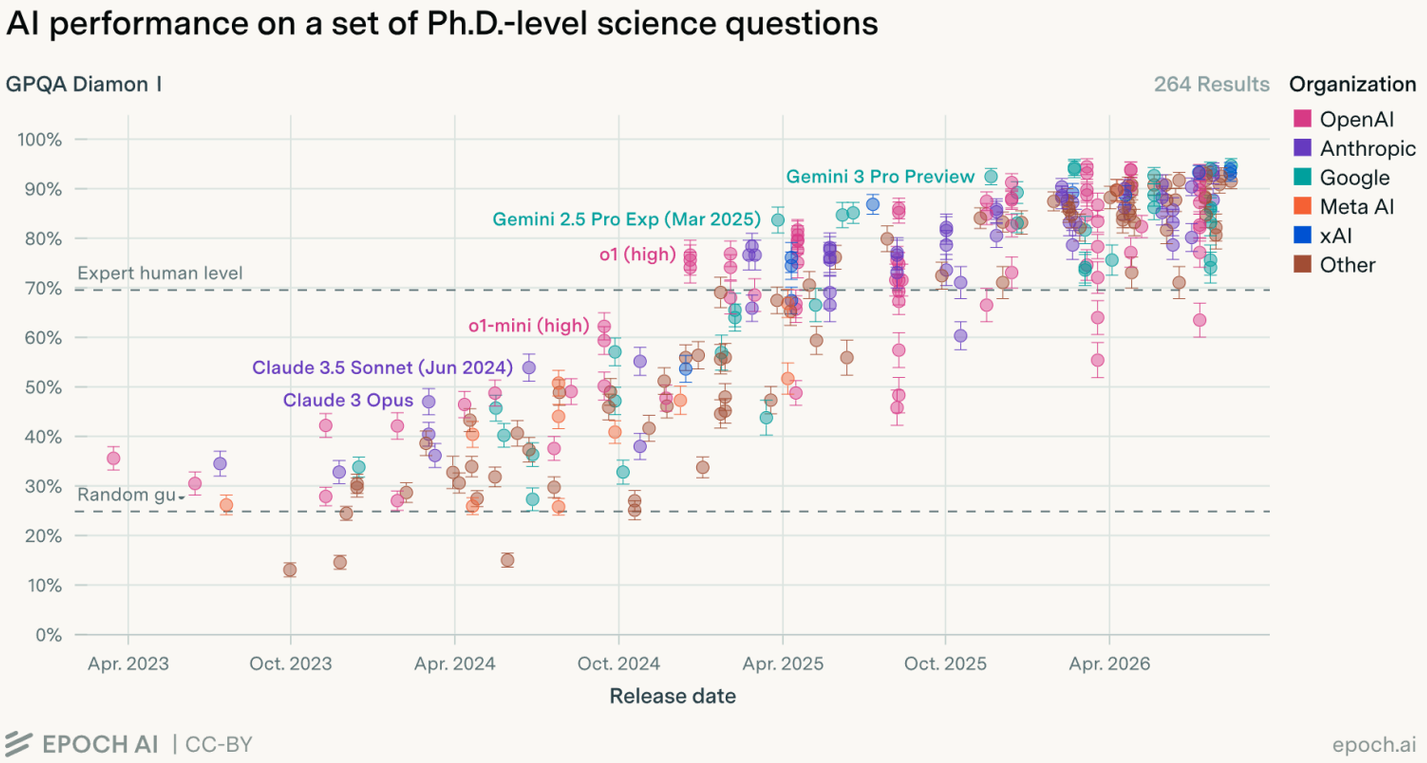
2 · The landscape moved from chat to agents

Three generations in four years: chat, reasoning, agents



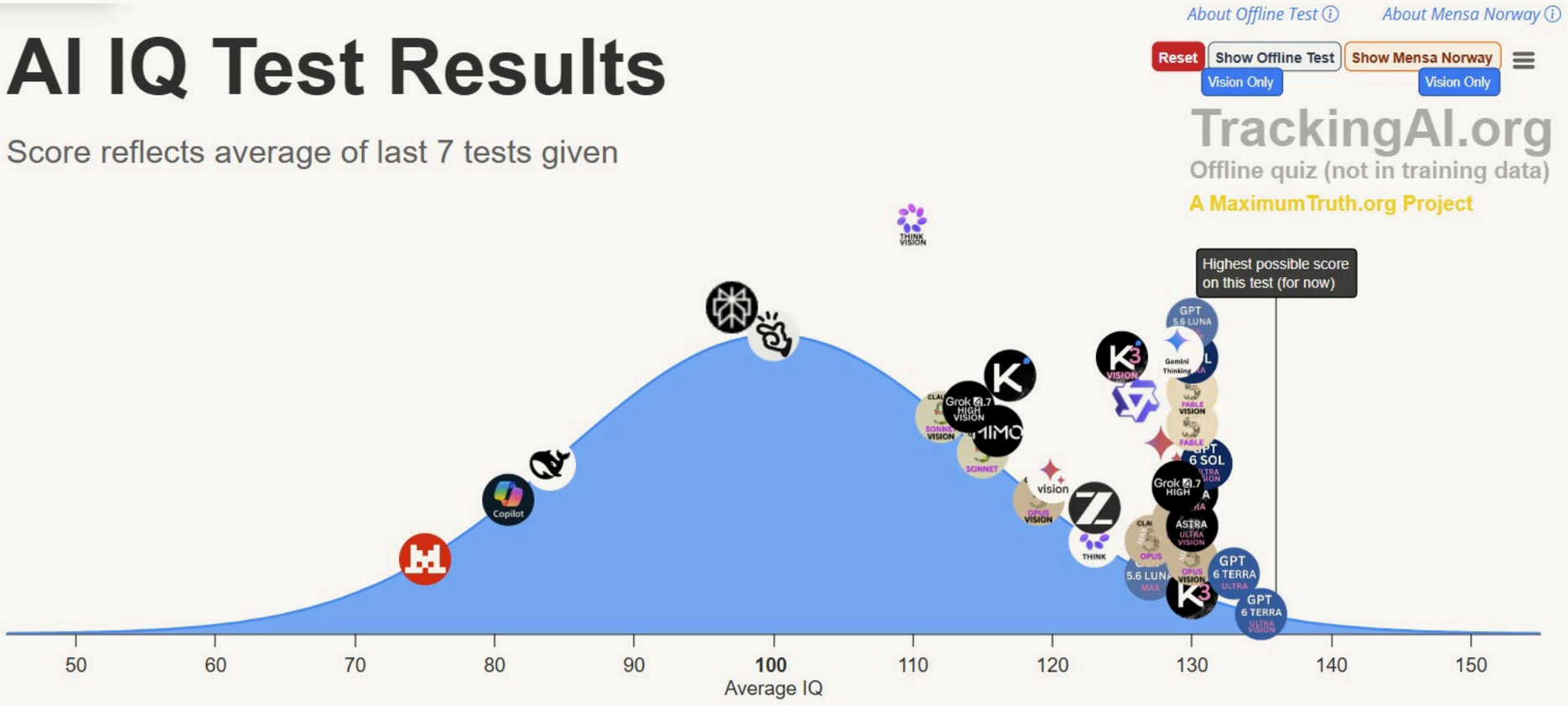
After Mollick (2026), Korinek (2025) and Velikov (2026).

On PhD-level science questions, models are now above expert level



GPQA Diamond accuracy by model release date, 264 results; each dot is one model. Dashed lines: random guessing (25 percent) and expert human level (about 70 percent). Epoch AI, CC-BY.

Models differ



Agents raise econometric coding success from 74 to 96 percent

Galiani et al. (2026) benchmark applied econometric tasks in Stata, R and Python. Moving from a chatbot that writes one script to an agent that runs and revises its own code is what raises task success.

Under the chatbot, results differ a lot by software. Under the agent, the differences largely disappear.

74 → 96%

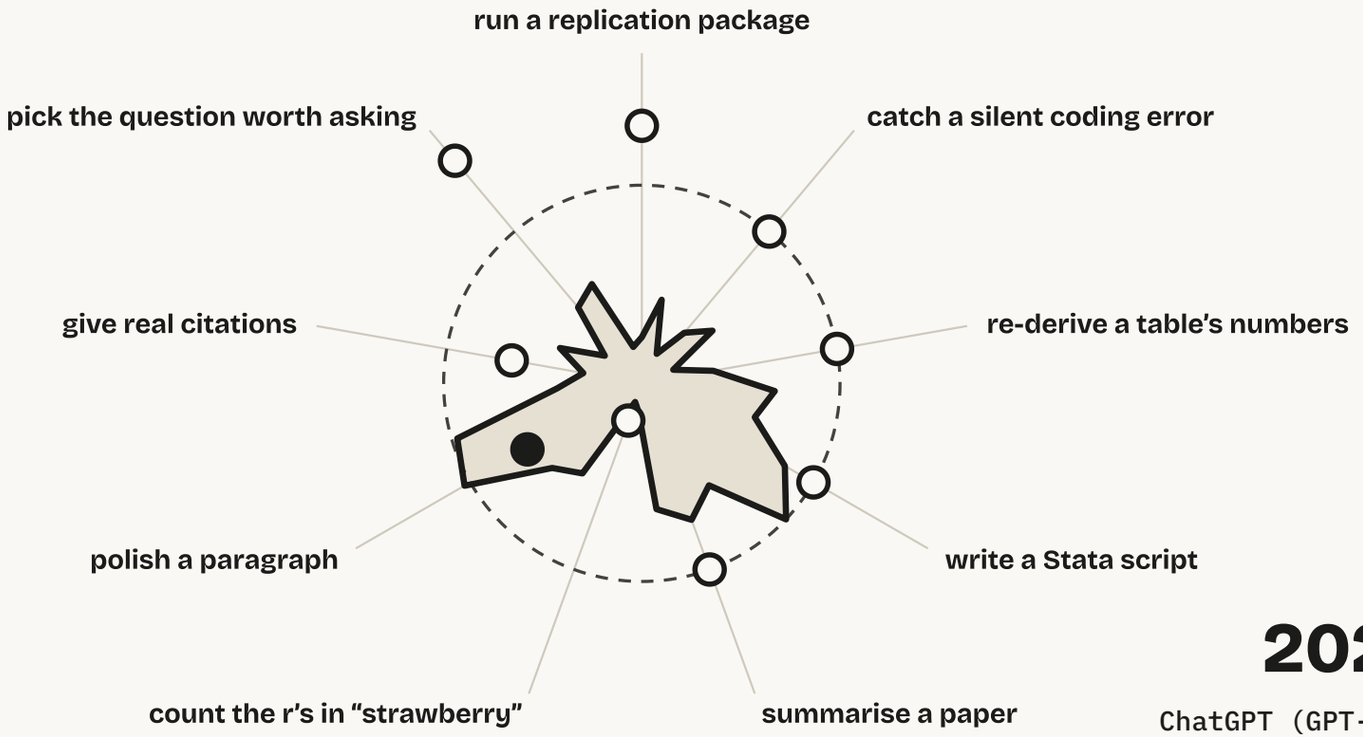
task success, chatbot to constrained agent

8 cents

extra cost per run

Galiani et al. (2026), NBER working paper 35588, abstract. “Constrained agent”: one that runs its code and fixes errors, but within a fixed set of tools.

But the frontier is jagged



2022

ChatGPT (GPT-3.5)
1 of 9 tasks inside

inside the frontier: the AI does it — the frontier in earlier years - - tasks equally hard for a person

Next year | 2022 | 2023 | 2024 | 2025 | 2026

ChatGPT polishes a paragraph. It cannot take in a whole paper, invents its citations and miscounts letters.

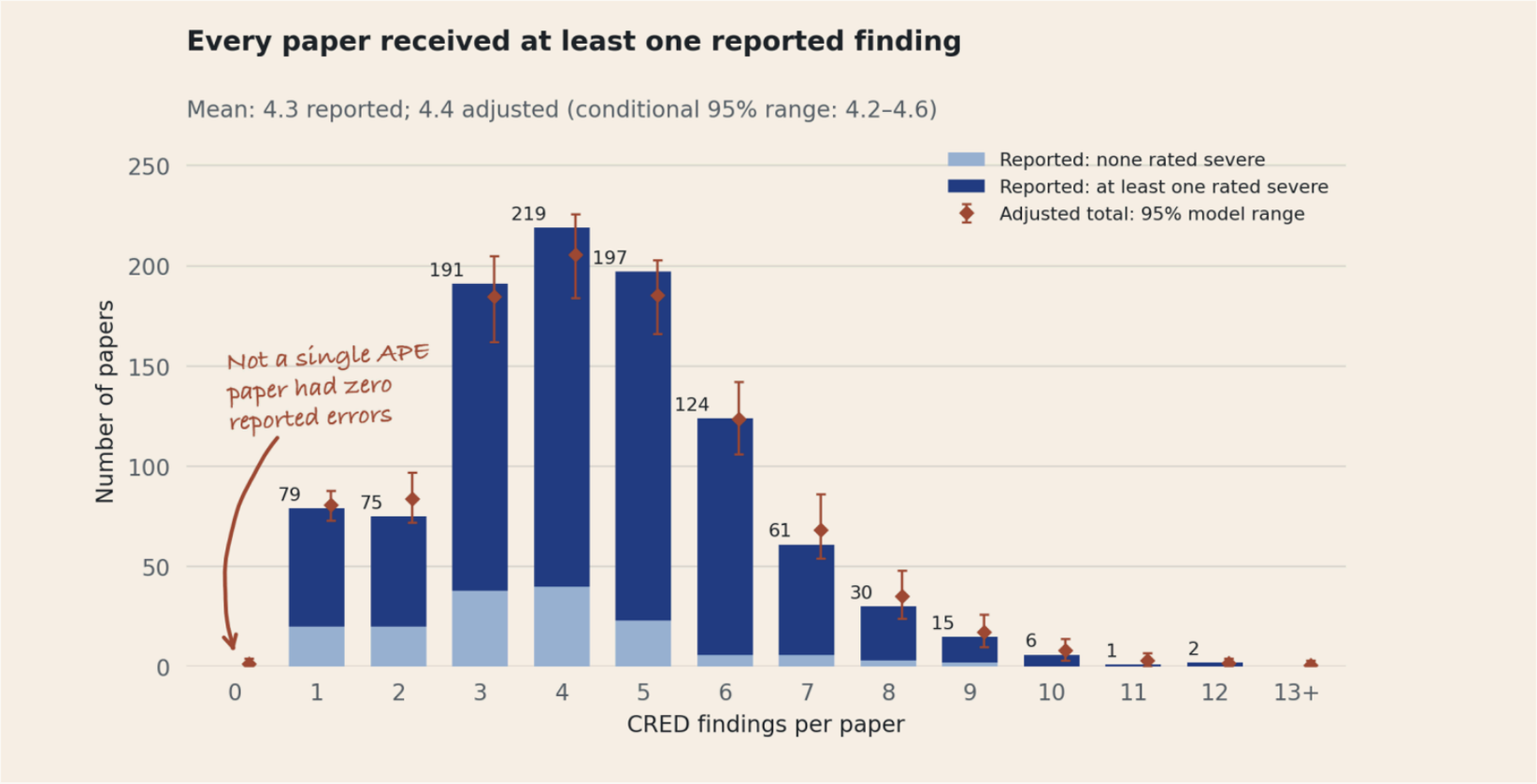
Stylised, after Mollick (2023) and Dell'Acqua et al. (2026). Distance from the centre is how hard a task is for a person. Which year each task moves inside is the deck's reading of model releases, not a measurement.

Efforts underway to automate research

1. **Project APE** has automated 1,000 policy-evaluation papers.
2. **Alejandro López-Lira** built a system that finds a problem, generates a theory, verifies it adversarially and writes a publication-ready paper.
3. **Novy-Marx and Velikov (2026)** describe how academic research could be automated with LLMs, and what that does to the literature.

Links to Project APE and López-Lira's system are on the workshop resources page.

Are we there yet?



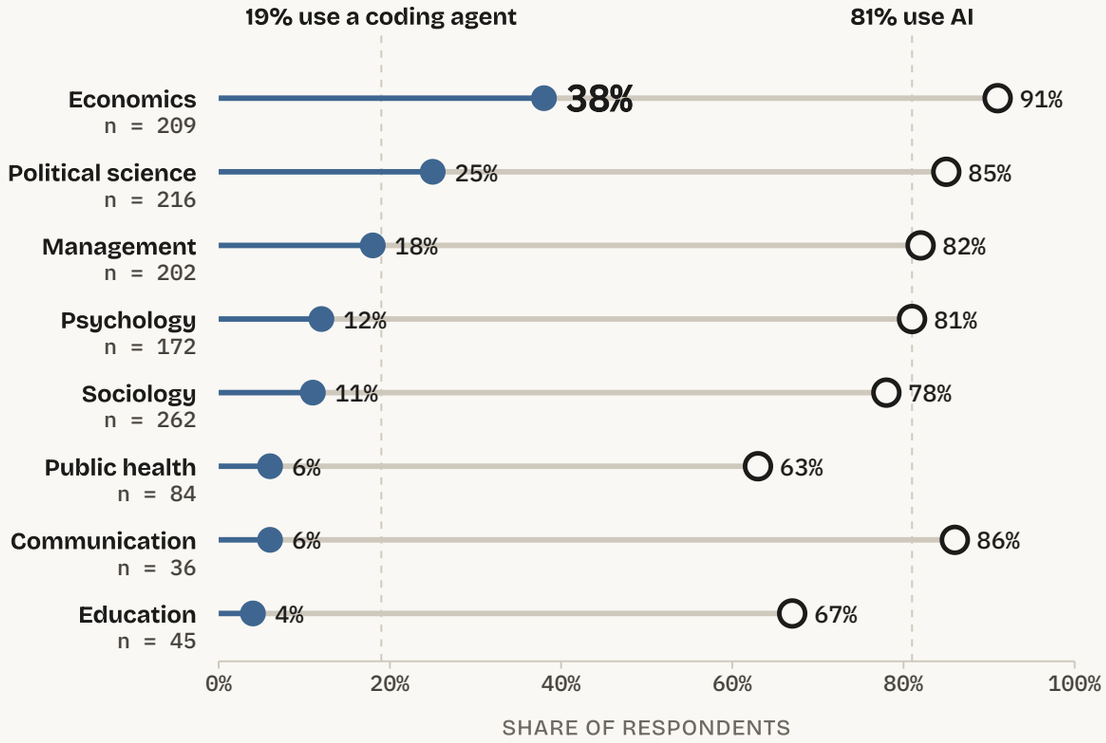
“We conclude that, in the first half of 2026, our autonomous policy evaluations remained unreliable despite our best efforts with frontier models and research workflows designed for reliability.”

APE Briefs, October 2026

“More capable foundation models, or better verification tools feeding back into the research process, may change that.”

APE Briefs, October 2026

Most social scientists use AI, few use an agent (from February/March)

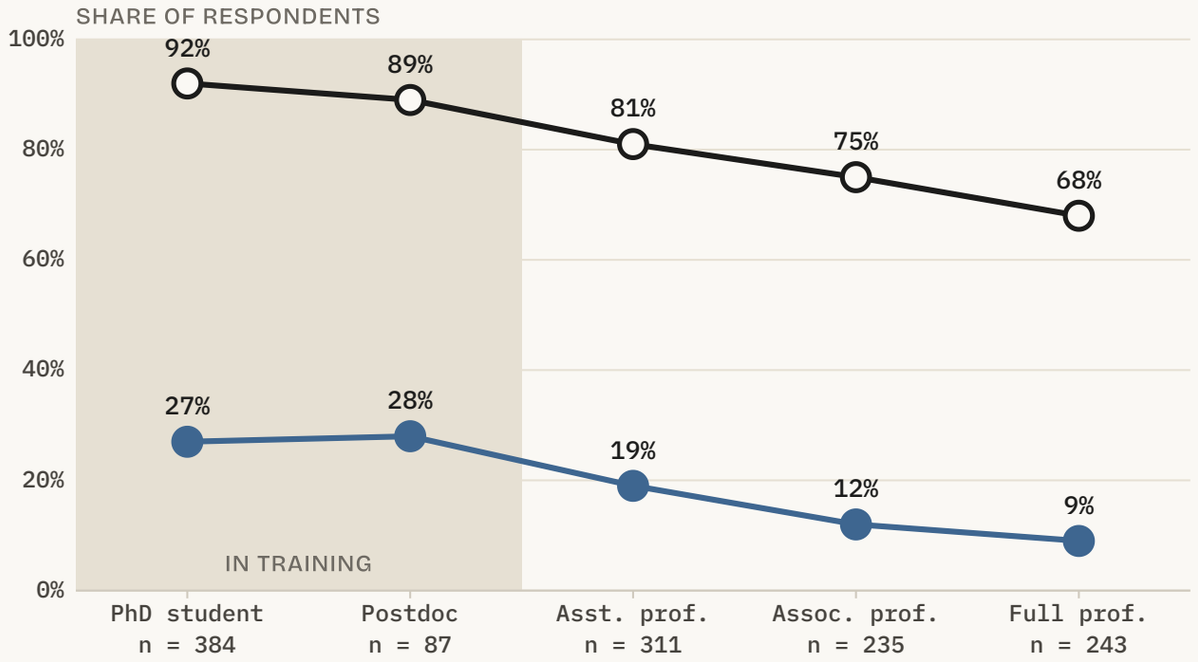


uses a coding agent uses any AI tool

Filled: uses a coding agent. Open: uses any AI tool.

Measured. Lyttelton, Massenkoff and Wilmers (2026), Anthropic Research survey of 1,260 social scientists, February–March 2026. Self-selected respondents, so the levels are likely upper bounds; the ordering across fields is the point.

Adoption falls with seniority



uses a coding agent uses any AI tool

Measured. Lyttelton, Massenkoff and Wilmers (2026), 1,260 social scientists, February–March 2026. The shaded band marks the two stages in training; it is the deck's annotation, not the survey's.

Check-in: the website

3 · Use cases for AI

Three jobs for an agent

1 · Research

- Literature-review summaries
- Pre-submission referee reports
- Feedback on your own drafts
- Responses to referees

Mental model: a feedback machine.

2 · Coding

- Stata, R and Python scripts
- Debugging error logs
- Robustness checks
- Replication packages

Mental model: a talented RA.

3 · Practical and admin

- Update slides and materials
- Mock exams and problem sets
- Conference budgets, CVs
- Websites for a paper

Mental model: a personal assistant.

Admin is an important use case

Friction is highest. The cost of starting dominates the cost of doing.

The quality bar is lower. No referee, no audience: a funder checks the contents.

There is nobody else to delegate to. Small tasks are not worth giving to someone else.

Example: a CV for research grants

Different funders, different required formats.

Without an agent. Reformat, re-order, drop sections, rewrite the bio.

With an agent. Drop the master CV and the funder's template in the folder. The agent reformats, you check.

A task that is easy to verify and that nobody pays you for.

AI is going to change how we do research

Same research, done faster

Faster coding. Stata, R and Python drafts, debugging error logs, robustness checks, replication packages.

Better writing. Structured critique, text edits, detailed feedback

New outputs, new projects

New ways to communicate a paper. An interactive website, a policy brief, a dashboard or an audio summary, all from the same paper.

New kinds of projects. Lower costs for trying out new things, mass automation (Project APE), automated replication and extension (Schwartz et al. 2026), AI agents as simulated humans (Expected Parrot).

AI use cases in research

1 · Coding

- Faster results
- New types of projects
- Websites, interactive tools.
- API calls to download data.

2 · Ideation and feedback

- Direct feedback on drafts
- Thinking through ideas
- New types of projects
- Consistency between paper and code.

3 · Writing and presentations

- Editing prose
- Grammar and spelling
- Consistency between different parts of a paper
- New ways of making slides

**AI should let us do new things,
not only old things faster**

AI rewards expertise

What the expert brings

Better questions. An expert asks the question that is worth asking.

Steering. An expert knows which follow-up moves the conversation where it should go.

Validation. An expert can tell when the output is wrong.

What the tool does not

It does not supply the question. It will just do what you ask for.

It does not replace judgment. You decide whether it is worth doing.

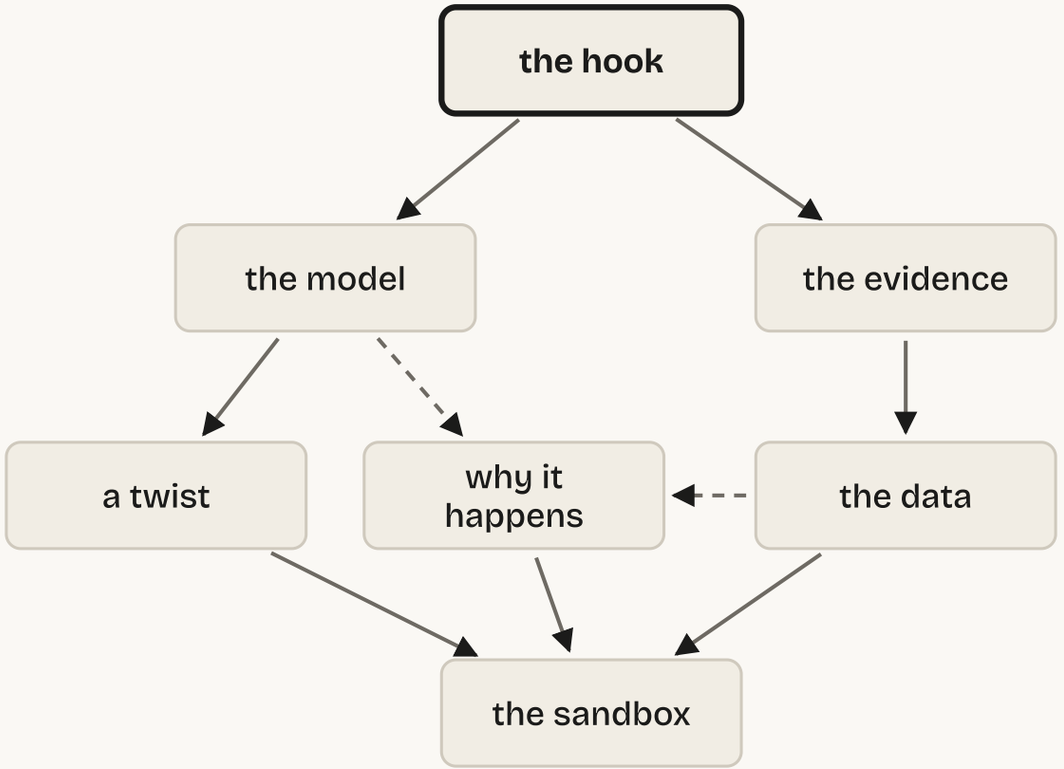
It does not know your institution's rules unless you put them in the context: the journal's AI policy, the funder's format.

Different ways to explain your work

`/explorable-explanation` builds a website as a non-linear tree of short HTML decks, one idea per slide, with forks placed where the reader becomes curious.

How it works

1. Reads the source material, then writes `plan.md` like a script: the hook, what the reader sees and does, and where the branches go
2. Shows you the first batch before it builds the rest
3. Screenshots every slide and reworks the ones that break the craft rules
4. Sends adversarial subagents to judge writing, interface, visuals and teaching
5. A “creative director” subagent gives it a visual identity; an impatient-reader subagent flags every slide where the flow breaks



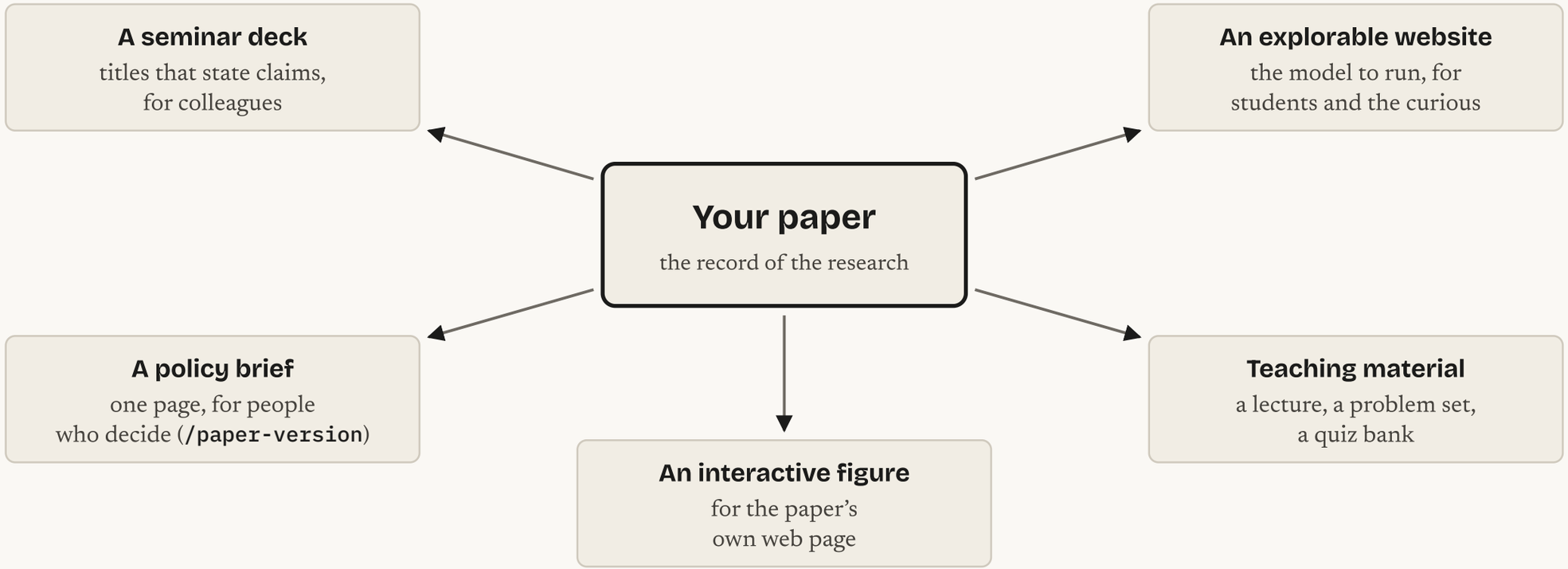
Each box is a short deck. Dashed arrows are forks the reader chooses from inside a slide.

Some examples

Explorables: Housing return paper and Financial advice from LLMs. Created using Nityesh Agarwal's skill for explorable websites, `explorable-explanations`.

Slides: Housing-returns paper and Financial advice from LLMs. The skill: github.com/claesbackman/AI-research-feedback, `Skills/explorable-deck`. Adapted from Nityesh Agarwal's skill for explorable websites.

We do not have to be bound by the PDF



Innovations are already happening

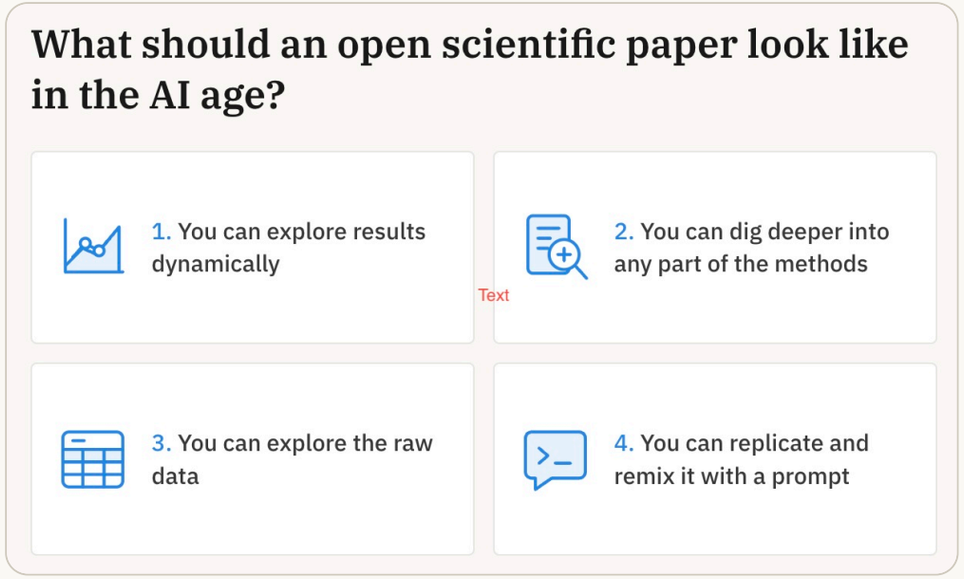
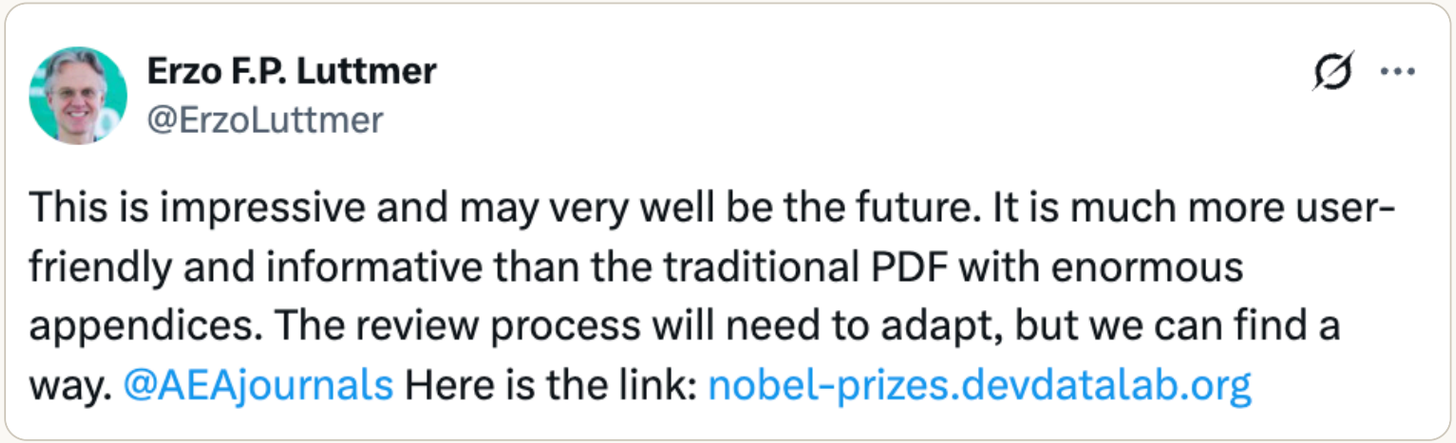


Figure from https://nobel-prizes.devdatalab.org/

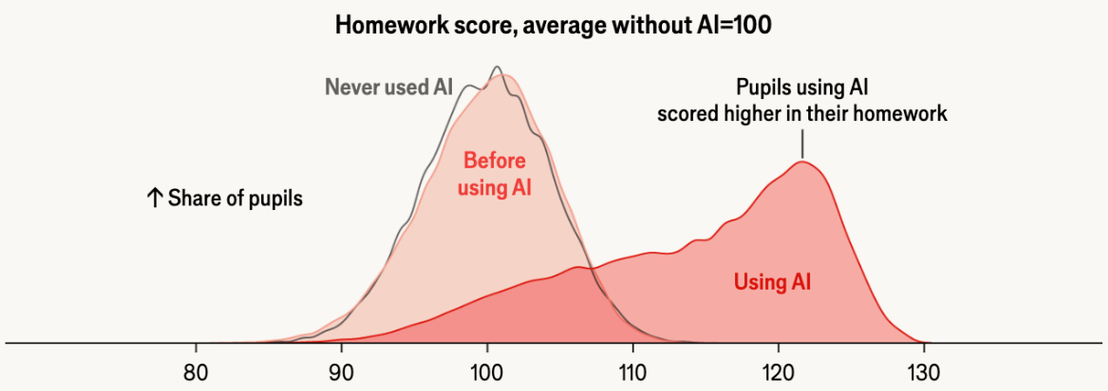


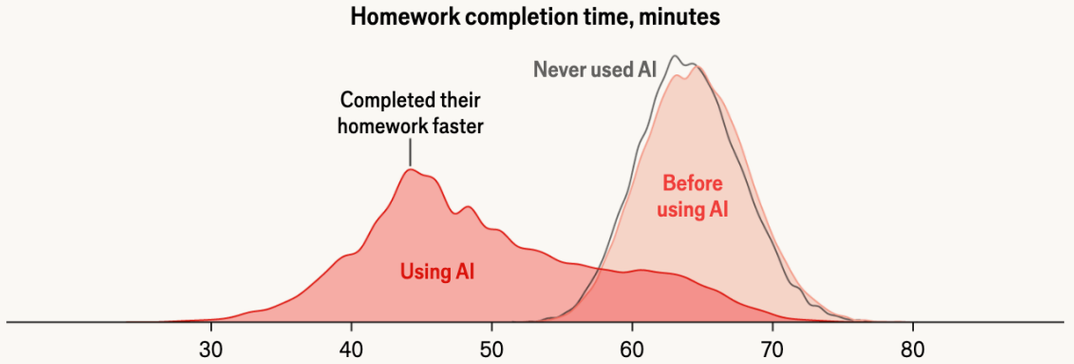
4 · Teaching in times of AI

The challenge with AI: take-home grades stopped measuring learning



With AI, homework got better and faster





+18%

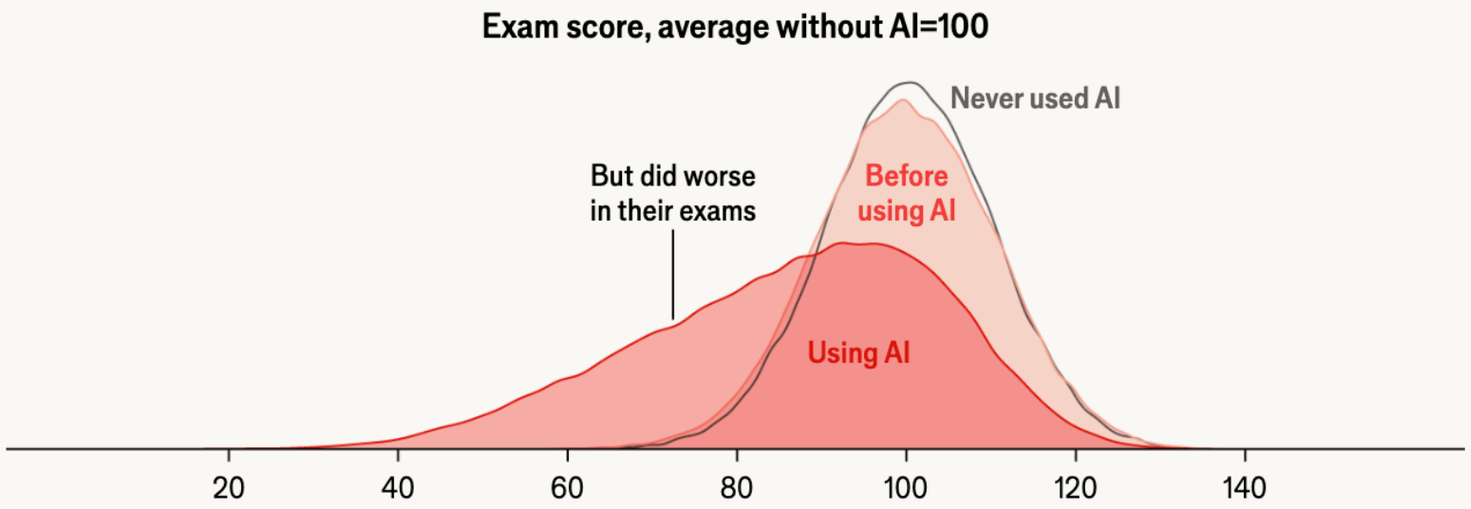
homework score for pupils using AI

−30%

homework completion time

Distributions for pupils in China: never used AI, before using AI, and using AI. Charts reproduced as published; study as cited in Bryan (2026).

But exam scores fell



−20%

exam performance for the same pupils

Homework up, time down, exams down. The homework stopped teaching.

Same study and source as the previous slide.

AI breaks the grade–knowledge link

1. **The grade–knowledge link is broken.** A good take-home grade no longer shows what the student can do
2. **Students need to use AI well later.** Banning it everywhere leaves them untrained.
3. **AI should let us teach more.** Tutoring, feedback and individual practice used to be too expensive.

Bryan (2026), kevinbryanecon.com/teachwithai.

Protect learning: decide, test, do not assign what AI can do

1. **Decide what should be learned without AI.** Be explicit about which assignments test AI-independent competence.
2. **Give students an incentive to learn.** A closed book exam gives incentives to learn.
3. **Do not give AI-cheatable assignments.** Almost every take-home assignment can be done by a frontier model today.
4. **Give students an idea of how AI impacts learning.** Everyone is struggling through what AI means for teaching, including students.

1, 2, 3 from Bryan (2026). 4 is mine.

Then teach more with AI

For the students

Help them learn more efficiently. Class-specific tutoring, spaced repetition, mastery learning at scale.

Personalise assignments. Problems at the level each student is struggling with.

Rules 4–7 of Bryan (2026).

For the teacher

Improve your own teaching. Find out where you were confusing before the exam tells you.

Raise standards. If research and drafting are cheaper, the bar moves up.

Three things to build: a tutor, a misconception digest, levelled problem sets

Course tutor

A shared assistant (a Claude project, custom GPT or NotebookLM) loaded with *your* lecture notes and problem sets.

Standing instructions: use the course's notation, never give the answer, ask the student to explain first, return to topics they missed.

Misconception digest

Each week, export the questions students asked, anonymised.

Prompt: "Group these by underlying misconception, rank by frequency, and point to the lecture slide most likely to have caused each."

Levelled problem sets

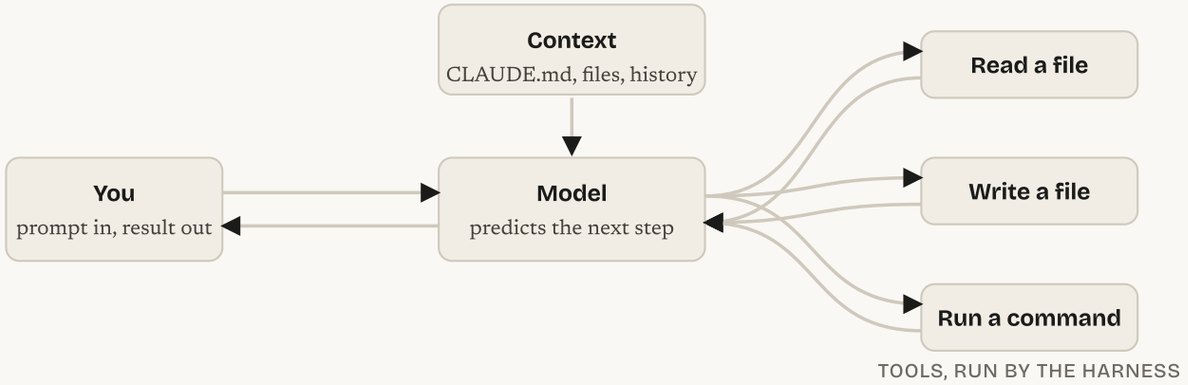
One problem, three difficulty tiers. Assign the tier by last week's quiz result.

Prompt: "Write this problem at three levels: scaffolded, standard, extension. Same concept, full solutions, and a rubric."

Student questions are personal data. Anonymise before exporting and check the institutional policy once.

5 · Tools, context, and workflows

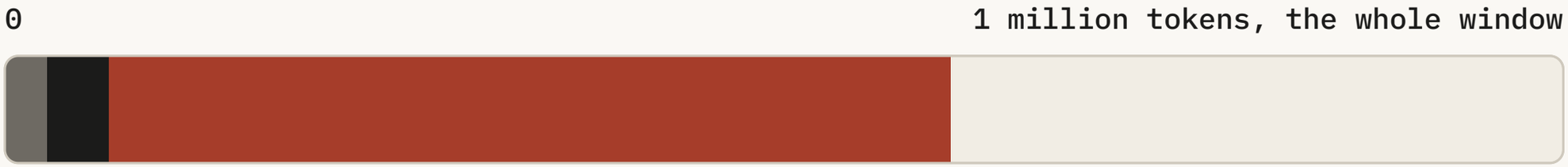
The agent is a loop: prompt, model, tool, result, repeat



Step Reset Press **step** to run the request through the loop.

Context is a budget

Everything the model conditions on has to fit in one window: the standing instructions, the tool definitions, the conversation, and every file it reads. Drag the slider to the right.



system prompt, CLAUDE.md, tool definitions conversation so far files the agent has read

PDF pages read this session 1200

1,200 pages, about 6 long PDFs ≈ 540,000 tokens, 61% of the window used in total.

Stylised arithmetic with real orders of magnitude: about 1 million tokens in the window, about 450 tokens a PDF page, so a 200-page PDF is 80–100k. The fixed segments are rules of thumb for a Claude Code session. Quality degrades well before the bar is full.

Four good habits

Point at files	`@filename`	Name the file you mean; do not let the agent read everything in the folder.
One task, one session	`/clear`	Start a new session for a new task. Thirty turns or a shifted task: restart. Signs of drift.
Watch the meter	`/context`	Shows what is filling the window and by how much.
Convert once	`/pdf-to-markdown`	Long PDFs become markdown once, up front; the markdown is committed and pointed at from then on. Which formats need it.

The commands are Claude Code's; Codex has equivalents. The starter CLAUDE.md template is in the appendix.

Five prompting rules: specific, iterate, correct early, verify, restart

1

Specific

“Feedback on the identification strategy in Section 4”, not “feedback on my paper”.

2

Iterate

Ten to twenty turns, not one. Each turn adds context.

3

Correct early

If the first answer is off, fix it now, or restart.

4

Verify

Run the code. Re-read the passage. Check the citation.

5

Restart

Thirty turns or a shifted task: a fresh session.

Providers trade places monthly; switching costs are low

The frontier labs

Claude, ChatGPT and Gemini have each led at some point in the last year. Grok (xAI) and Llama (Meta) compete. The leading provider changes with each release.

Open weights

Cheaper to run, and many are not far behind on performance. Mistral is European, which can matter when a data-protection office asks where the model runs.

Switching costs are low. It is more important to start.

Pro at $20 a month is enough to start

Free	`$0`	Casual chat on claude.ai; no real Claude Code usage
Pro	`$20 / month`	Light daily Claude Code use, projects, research **START HERE**
Max	`$100–200 / month`	Heavy Claude Code use (5× or 20× the Pro limits); the frontier model all day
API	`pay per token`	Automation, batching, custom-built tooling

Claude's tiers as of August 2026; OpenAI's are similar.

The default model is enough for daily work; step up for review

Fast, cheap	`Haiku`	Conversions, summaries, bulk tasks
Default	`Sonnet`	Daily coding, editing, drafting
Frontier	`Opus`	Hard reasoning, adversarial review
Most capable, slowest	`Fable`	Long-running agents, the hardest tasks; costs and waits more

Claude's model tiers as of August 2026.

Effort spends the same token budget as files

Effort sets how many tokens the model spends thinking before it answers. It is a setting in the tool, next to the model choice.

Turn it up for deep analysis and adversarial review. Turn it down for speed on routine tasks.

3–4

characters of English per token

15–30k

tokens in a short paper

Tokens are the unit models bill and reason in. Thinking tokens and file tokens come out of the same window.

Rules of thumb from the source deck.

Chat runs in a sandbox; an agent runs on your computer

Chat (ChatGPT, Claude.ai)

- Runs on the provider's server, walled off from your machine
- Sees only what you paste or upload
- Gives text back; you move it where it belongs
- Answers get generic when the paste is thin

When to use: brainstorming, polishing one paragraph, quick lookups.

An agent (Claude Code, Codex)

- Runs in a folder on your computer
- Can read any file in the project, and edit it
- Runs code, shell commands, search, web, MCP tools
- Output is grounded in your files and lands in them

When to use: any task that touches more than one file or needs to execute code.

But the two are converging

What chat borrowed from agents

Claude and ChatGPT both have **projects**: upload files, set standing instructions, keep a memory per project. That is the context half of what an agent has, in a chat window.

Both apps now contain an agent: the *Code* and *Cowork* tabs in the Claude app, Codex inside ChatGPT.

What chat still lacks

The loop. Projects hold context; they do not read your folder, write files, or run code and fix what broke.

The workflow is the same regardless of the software

SURFACE	WHAT IT IS GOOD FOR
VS Code	What I use. File tree, diff view, integrated terminal, side-by-side PDF preview.
Terminal	`claude` or `codex` in any shell. Good for servers and quick one-shots.
Desktop app	Claude Desktop's *Cowork* and *Code* tabs, or the Codex desktop app.
Web, cloud	claude.ai/code or chatgpt.com/codex. Point at a GitHub repository.
Mobile	Capture ideas; monitor or redirect cloud agents. Not for editing.
Other editors	Cursor, JetBrains, Zed. Keep your editor, add the agent.

More at claesbackman.com/agentic-ai-overview.

Claude Code vs. Codex: two names for the same workflow

Both will read your files, run code, edit drafts, iterate, and hand sub-tasks to copies of themselves. A *skill* is a saved prompt invoked by name.

Claude Code (Anthropic)

Standing instructions: `CLAUDE.md`
Skills trigger: `/review-paper`
Backed by: Fable, Opus, Sonnet, Haiku
Surfaces: VS Code, CLI, web, desktop

Codex (OpenAI)

Standing instructions: `AGENTS.md`
Skills trigger: `$review-paper`
Backed by: GPT family
Surfaces: VS Code, CLI, web, mobile

The difference between tools is how easily you supply context

For prompting: make a plan, documents the work, and get something you can check

> *Using the ECB Data Portal API, download two quarterly series from 2000 onwards: the euro-area residential property price index and the deposit facility rate averaged to quarterly. Look up both series keys first and show me the keys and your planned requests before you download anything.*
>
> *Save the merged series as* `data/raw/ecb_hp_rate.csv` *and write* `data/README.md` *giving each series key, its units, and its transformation. Use Python.*
>
> *Then plot the two series together, and report the 2008Q1 and 2022Q1 values of each so I can check them against the portal.*

Three paragraphs, three moves: plan first, document, verify. What the prompt is testing for is in the appendix.

Six things to try this week

Explain a paper

And how it is relevant for your own work. Attach the PDF.

Fetch data

From a public API: ECB, FRED, Eurostat, World Bank. Plan first.

Write code

For a paper, one step at a time, running each step.

Update teaching materials

Slides with 2015 examples get current ones; the structure stays.

Edit your writing

In your own voice.

Mock exams and problem sets

Your students may already be doing this.

6 · Verification

You are responsible for the output, and so you have to make sure that it is right

Agents are fast, cheap and competent at code, including in languages you do not know.

> **The errors that matter are not code crashes**
>
> Agents are very good at finding and fixing what crashes. Had an agent used the wrong fixed effect in your table, the code would still run and look plausible, but it would be wrong.

claesbackman.com/verifying-llm-output; Litt (2026); Goldsmith-Pinkham (2026); Scott Cunningham.

First principle: read whatever goes into your paper

The code is the analysis. Once a result is in the paper, you are responsible for it.

Not all code is equal. You don't need to know the website code, but you should probably check the regressions.

A diff is a smaller thing to read than a codebase

A reviewer that sees everything

Spreads its attention thin, gives generic answers, and spends the window on files that did not change.

A reviewer that sees the diff

Knows exactly what is new and compares it against code you already checked.

You can use `git diff`, or your agent can. A *diff* is the list of lines that changed since the last saved version.

Version control is a verification tool. This is a good reason to use Git.

Different levels of checks

CHECK	WHAT YOU DO	WHAT IT CATCHES	COST
1 · Fresh agent reviews the diff	A fresh session or subagent reviews the diff and reports	Errors invisible to the agent that wrote the code	Minutes
2 · Different model	Same diff, model from another developer (Codex vs. Claude)	Shared habits: two copies of one model make the same mistakes	Minutes
3 · Make the agent quiz you	Fresh session explains the change, then quizzes you	Your own understanding gap	10–15 min
4 · Reimplement in another language	Fresh folder, `spec.md` and `README.md` only, other language and model, you run it	Code that faithfully implements something other than the paper	Hours

Cheap ones first, the expensive one for the main tables. Checks 2 to 4 each have a room in the appendix; click the name.

Four habits for text output: run, check, read, re-read

HABIT	WHAT TO LOOK FOR	HOW
Run the code	Plausible Stata, R or Python that does not always run	Run it yourself
Check the numbers	Statistics, dates and quotes get invented	Re-derive from the table or the paper
Read the citations	Citations to papers that do not exist are common	Look for it on Google Scholar
Re-read the passage	Confident descriptions of text that is not in the draft	Open the file, read the section

Adapted from Korinek (2025).

7 · What this means for academic work

Friction drops, the bottleneck moves, verification is key, things will keep moving

1. **Friction drops** in writing, editing and admin: the paper website, the grant CV.
2. **The bottleneck shifts** from producing to judging.
3. **Verification becomes the differentiator.** Not typing speed, not LaTeX fluency.
4. **Tooling will keep moving.** The principles, specificity, iteration, verification, will not.

Disclosure and data norms are still in flux

Disclosure

- Journals mostly require acknowledging AI assistance; the specifics vary
- Briefs, teaching and internal documents have no strong norm yet
- Default rule: disclose where your name is on it

Data handling

- An agent sends the files it reads to the model provider, not only your prompt
- Confidential microdata, HR material and DUA-covered data stay out until the institution says otherwise
- Web chat and a local agent are different tools with different policies

Publishing gets noisier; research can get better

Publishing gets noisier

Journals fill with polished, competent-looking mediocrity. Signal-to-noise drops and reviewing gets harder. Editors and referees absorb the cost.

Research can still get better

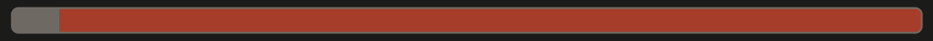
Ideas that could not be written before reach readers. The language barrier falls. Feedback that used to need a good department is available to anyone with a folder and twenty dollars.

End of Session 1

Later we will use agents together.

Hands-on with your own materials: a referee report on your draft, a voice file from your own writing, your code and your paper side by side.

Bring a draft you care about and four or five PDFs of your own writing. Slides, skills, guides: claesbackman.com/ai_workshop



Claes Bäckman, SAFE

Appendix

FROM	SLIDE	WHAT IS IN IT
5 · Tools	CLAUDE.md starter template	Standing instructions to adapt for your own project
5 · Tools	The six tools the harness adds	Read, edit, shell, search, web, MCP
5 · Tools	Signs of session drift	When to start a fresh session
5 · Tools	Which file formats need converting	Text, convert once, read through code, OCR
5 · Tools	Converting is one sentence	A prompt that converts a folder of PDFs
5 · Tools	What the ECB prompt is testing for	APIs, series keys, and what to check
6 · Verification	My skill library	Nine skills on GitHub, free to fork
6 · Verification	Four checks, the full table	What each check catches and what it costs
	References	Everything cited on the slides

CLAUDE.md starter template

```
# About me
I am a researcher in [field].

# How I want you to work with me
- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md).
  No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

# Things to avoid
- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.
```

Always loaded, every session. Standing instructions, the things you would otherwise repeat in every prompt.

Day 2 builds one for your own project with `/init`.

Back to the harness, the habits or the appendix overview.

The harness adds six things the model cannot do alone

CAPABILITY	WHAT IT LETS THE AGENT DO
Project files (`Read`)	Read files, inspect folders, understand the project structure
Editing (`Edit`, `Write`)	Propose and apply changes; create or modify files
Shell (`Bash`)	Run commands such as `Rscript`, `pdflatex`, tests, linters
Search (`Grep`, `Glob`)	Search file names and file contents in the project
Web (`WebSearch`)	Search or fetch web pages when web access is enabled
MCP and plugins	Connect to external tools (GitHub, databases, custom servers) through a standard interface, the model context protocol

The names are Claude Code's; Codex has the same six under other names. Back to the loop or the appendix overview.

Signs of session drift

1. Claude ignores `CLAUDE.md`
2. Repeats mistakes you already corrected
3. Invents files that do not exist
4. Gives circular answers
5. Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

Thirty turns, or the task has shifted: start fresh. Always.

Back to the habits or the appendix overview.

Text formats are free; binaries cost a conversion

Native, no friction	`.md`, `.tex`, `.txt`, `.bib`, `.csv`, `.json`, `.yaml`, and code: `.py`, `.R`, `.do`, `.ipynb`
Convert once	`.docx`, born-digital `.pdf`, `.xlsx`: to markdown or CSV, saved next to the original
Read through code, not as text	Data binaries such as `.dta` and `.rds`: the agent loads them in Stata, R or pandas and looks at the output
Hard: OCR or skip	Scanned PDFs, survey exports such as `.qsf`

Convert once, commit the markdown alongside the original, point the agent at the markdown.

Back to the habits, on to converting, or the appendix overview.

Converting is one sentence to the agent

> “Convert every PDF in `papers/` to markdown, save each next to the original, and tell me which ones came out garbled.”

Back to the file formats or the appendix overview.

What the ECB prompt is testing for

1. **What an API is.** A URL that returns data as CSV or JSON instead of a web page. The ECB Data Portal serves every published series with no registration and no key.
2. **The series key is the whole trick.** A series is addressed by a dotted key such as `RPP.Q.I9.N.TD.00.3.00`. The agent must look the key up before it can fetch anything, which is why the prompt asks to see the keys first.
3. **Same pattern elsewhere.** FRED, Eurostat, SNB, BIS, World Bank, OECD.
4. **Verify.** Agents invent series keys that look plausible and return nothing, or the wrong series. Check a few values against the portal's own interface.

Back to the prompt or the appendix overview.

My skill library

SKILL	USE
`/review-paper`	Pre-submission referee report, 8 agents
`/review-paper-light`	Fast 2-agent version, about 1 minute
`/review-paper-code`	Paper and code alignment review
`/review-grant`	Pre-submission grant review, 6 agents
`/review-pap`	Pre-analysis-plan review
`/audit-analysis`	Adversarial audit of changed analysis code
`/explain-diff`	Explain a code change as an HTML page with a quiz
`/paper-version`	Paper to policy brief, 1-page or 5-page summary
`/pdf-to-markdown`	Convert PDFs to readable markdown

All at github.com/claesbackman/AI-research-feedback (MIT), linked from claesbackman.com/ai_workshop. Fork them, modify them, make them yours. Back to check 3 or the appendix overview.

Check 1: a fresh agent reviews the diff, and reports

Two ways to get a reviewer with no stake in the code:

Ask the current agent to hand the diff to a subagent, a fresh copy of itself with an empty context.

Open a new session and point it at the diff.

> **Have the reviewer report, not fix**
>
> Look at what the agent found and verify whether you agree with them. .
>
> *Weakness: a fresh copy of the same model shares its habits. Hence check 2.*

Check 2: a different model breaks shared habits

Two copies of one model share training data and habits of thought, so they make the same mistakes. Send the same diff to a model from a different developer.

Check 3: make the agent quiz you

Open a fresh session and ask:

> *Read the last commit and the files it touches, then ask me three questions about what it changed. Do not tell me the answers until I have answered. If my answer is wrong, say so directly; do not tell me I was close.*

Always a new session, not the one that made the change. Starting fresh forces the agent to read the files instead of recalling the conversation.

A skill for it: `/explain-diff` writes one offline page about what changed, why, what it does to the results, and ends with a five-question quiz. More skills in the appendix.

Check 4: reimplement the analysis in another language

What and why

If your code is Stata, a second agent writes the results in Python or R from the paper's description alone. An idea from Scott Cunningham.

If coding slips are independent across languages, differences in the output reveal the errors.

How to isolate it

New folder, fresh session, a `spec.md` copied from the paper with the results deleted, a `README.md` describing the raw data.

This is a major enterprise, so reserve it for the main tables and for things that matter.

References

Bryan, Kevin A. 2026. *Eight Rules for Teaching in AI World*. Https://kevinbryanecon.com/teachwithai.html.

Galiani, Sebastian, Federico Ariel López, and Raul A Sosa. 2026. *AI Agents and Prompt Engineering in Econometric Coding*. National Bureau of Economic Research.

Mollick, Ethan. 2026. *A Guide to Which AI to Use in the Agentic Era*.

Novy-Marx, Robert, and Mihail Velikov. 2026. "Artificial Intelligence-Powered (Finance) Scholarship." *Journal of Economic Literature* 64 (1): 5–37.

Schwartz, Matthew, Isaiah Andrews, and Jesse M Shapiro. 2026. *An LLM Workflow That Reproduces, Improves, and Extends Published Economics Research*. National Bureau of Economic Research.

Velikov, Mihail. 2026. *EDHEC Master Class – Using AI for Research*.